Solution: Pangram

Problem Statement

Given a string `sentence` containing English letters (lower- or upper-case), return `true` if `sentence` is a **Pangram**, or `false` otherwise.

A **Pangram** is a sentence where every letter of the English alphabet appears at least once.

Note: The given `sentence` might contain other characters like digits or spaces, your solution should handle these too.

Example 1:

```
Input: sentence = "TheQuickBrownFoxJumpsOverTheLazyDog"
Output: true
Explanation: The sentence contains at least one occurrence of every letter of the
English alphabet either in lower or upper case.
```

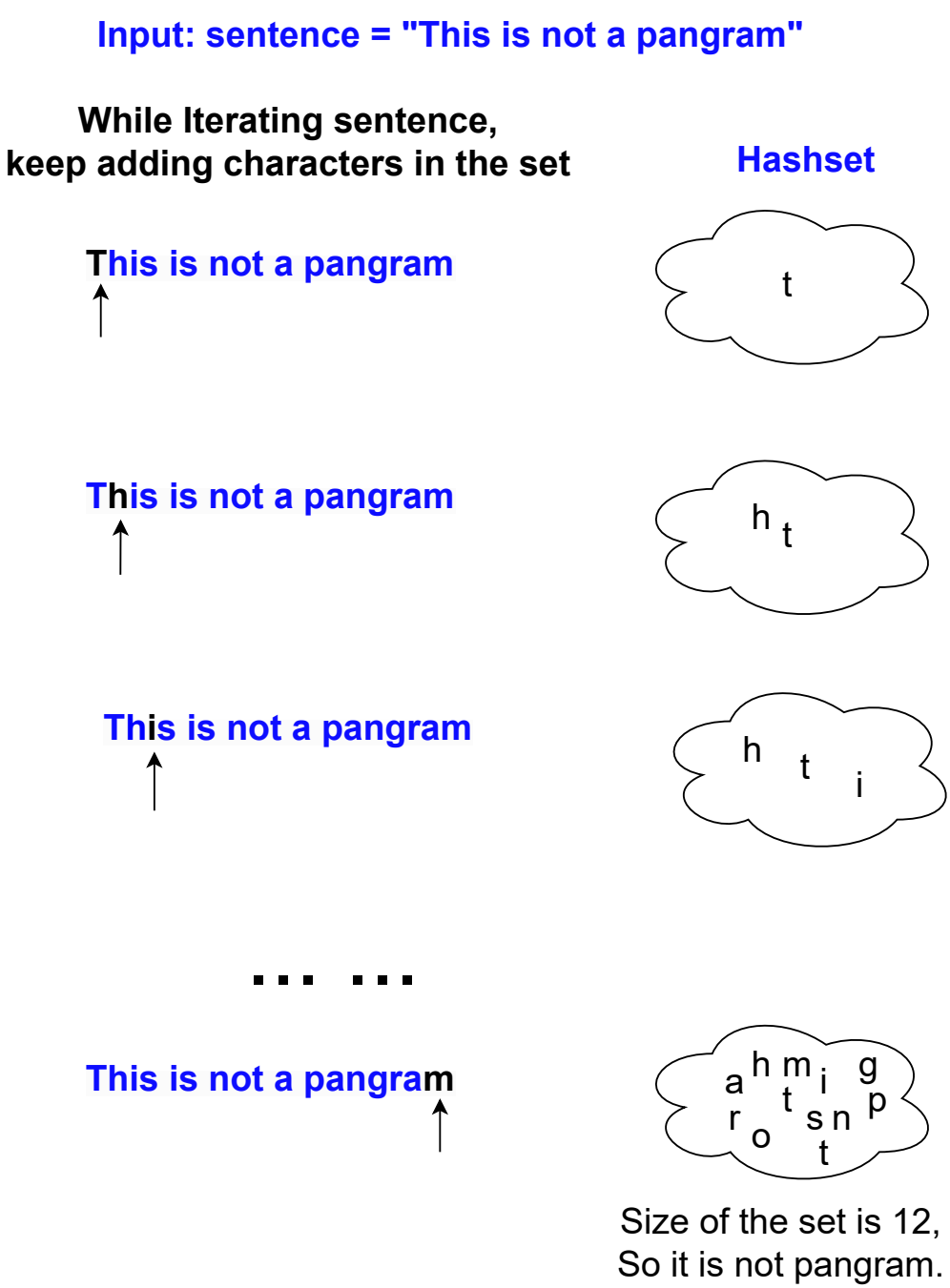
Example 2:

```
Input: sentence = "This is not a pangram"
Output: false
Explanation: The sentence doesn't contain at least one occurrence of every letter
of the English alphabet.
```

Solution

We can use a `HashSet` to check if the given sentence is a pangram or not. The `HashSet` will be used to store all the unique characters in the sentence. The algorithm works as follows:

1. Converts the sentence to lowercase.
2. Iterate over each character of the sentence using a loop.
3. Add each character to the `HashSet`.
4. After looping through all characters, compare the size of the `HashSet` with 26 (total number of alphabets). If the size of the HashSet is equal to 26, it means the sentence contains all the alphabets and is a pangram, so the function will return true. Otherwise, it will return false.



Code

Here is the code for this algorithm:

```java
import java.util.HashSet;
import java.util.Set;

class Solution {
  // Function to check if given sentence is pangram
  public boolean checkIfPangram(String sentence) {
    // Create a set to store unique characters
    Set<Character> seen = new HashSet<>();

    // Convert sentence to lowercase and iterate over each character
    for (char currChar : sentence.toLowerCase().toCharArray()) {
```

```
12            if (Character.isLetter(currChar)) {
13              // Add the character to set
14              seen.add(currChar);
15            }
16          }
17
18          // Return true if set size is 26 (total number of alphabets)
19          return seen.size() == 26;
20        }
21
22        public static void main(String[] args) {
23          Solution sol = new Solution();
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

1. **Iterating Over Characters**: The main operation in the code is iterating over each character in the input string. If the length of the input string is `n`, this iteration occurs `n` times.
2. **Set Operations**: For each character, the code performs a constant-time operation—adding the character to a HashSet if it's a letter. The time complexity for adding an element to a HashSet is typically $O(1)$.
3. **Overall Time Complexity**: Considering the iteration over `n` characters and constant-time set operations, the total time complexity is $O(n)$, where `n` is the length of the sentence.

Space Complexity

1. **HashSet Storage**: The HashSet `seen` is used to store the distinct characters encountered in the sentence. In the worst-case scenario, it will store all `26` letters of the alphabet.
2. **Constant Size Set**: Regardless of the input sentence length, the HashSet can only grow up to a size of `26`. This is because it only stores distinct English alphabet letters.
3. **Overall Space Complexity**: Given the HashSet's maximum size is constant (at most `26` characters), the space complexity is $O(1)$, meaning it is constant.

Conclusion

- **Time Complexity:** $O(n)$, where `n` is the length of the input string.
- **Space Complexity:** $O(1)$ (constant space, independent of input string length).

← Previous

Pangram (Easy)

Next →

Reverse Vowels (Easy)

Mark as Completed